

Summary

Dividing problems into smaller, self-similar problems is at the heart of recursion, making these divide-and-conquer algorithms especially suited for recursive techniques. In this chapter, we created a divide-and-conquer version of Chapter 3's program for summing numbers in an array. One benefit of this version is that upon dividing a problem into multiple subproblems, the subproblems can be farmed out to other computers to work on in parallel.

A binary search algorithm searches a sorted array by repeatedly narrowing the range to search in half. While a linear search starts searching at the beginning and searches the entire array, a binary search takes advantage of the array's sorted order to home in on the item it is looking for. The performance improvement is so great that it may be worthwhile to sort an unsorted array in order to enable a binary search on its items.

We covered two popular sorting algorithms in this chapter: quicksort and merge sort. Quicksort divides an array into two partitions based on a pivot value. The algorithm then recursively partitions these two partitions, repeating the process until the partitions are the size of a single item. At this point, the partitions, and the items in them, are in sorted order. Merge sort takes an opposite approach. The algorithm splits the array into smaller arrays first, and then merges the smaller arrays into sorted order afterward.

Finally, we covered Karatsuba multiplication, a recursive algorithm for performing integer multiplication when the `*` multiplication operator isn't available. This comes up in low-level hardware programming that doesn't offer a built-in multiplication instruction. The Karatsuba algorithm breaks down multiplying two integers into three multiplications of smaller integers. To multiply single-digit numbers for the base case, the algorithm stores every product from 0×0 to 9×9 in a lookup table.

The algorithms in this chapter are part of many data structure and algorithm courses that freshman computer science students take. In the next chapter, we'll continue to look at other algorithms at the heart of computing with algorithms that calculate permutations and combinations.